

Strukturalni paterni - ETFPay

U okviru ovog dokumenta opisano je 6 strukturalnih patterna, kao i njihove potencijalne primjene u sistemu ETFPay. Kao primjer za implementaciju na dijagramu klasa uzeti su Facade i Adapter patterni.

1. Facade (Fasada) Pattern

Izvršavanje novčane transakcije u sistemu zahtijeva interakciju sa više različitih klasa. Da bi se izvršio transfer bez fasade, klijentski kod bi morao samostalno pronaći pošiljaoca i primaoca, provjeriti stanje i validnost njihovih računa, kreirati novi zapis o transakciji te izračunati i ažurirati nova stanja. Ovakav pristup uzrokuje visoku povezanost klijentskog koda sa više različitih podsistema.

Fasada delegira pozive odgovarajućim podsistemima. Moguće je uvesti klasu TransferFacade, nakon čega klijent poziva samo metodu IzvrsiTransfer(posiljaocID, primaocID, iznos) dok fasada interno vrši manipulaciju instancama klasa Osoba i Racun, kao i spremanje u klasu Transakcija.

2. Adapter Pattern

Adapteri omogućavaju komunikaciju između nekompatibilnih interfejsa. Problem nekompatibilnosti između interfejsa klase Predlozak i interfejsa koji očekuje klasa Osoba se rješava klasom PredlozakAdapter. Ovaj adapter implementira očekivani PredlozakTarget, ali unutar sebe omotava instancu stvarne klase Predlozak. Kada klijent pozove metodu za izvršavanje pretplate ili kreiranje predloška, adapter taj poziv interno "prevodi" na odgovarajuće metode konkretne klase, odnosno klase Predlozak.

3. Proxy

Služi kao zamjena ili "posrednik" za drugi objekt kako bi kontrolirao pristup njemu, omogućio odgođenu inicijalizaciju ili dodao sloj sigurnosti. U našem sistemu, može se kreirati RacunProxy koji provjerava atribut Uloga prije nego što dozvoli uvid u stanje ili izvršenje isplate. Također, može se koristiti za učitavanje na zahtjev duge historije transakcija radi uštede memorije.

4. Decorator

Omogućuje dinamičko dodavanje novih funkcionalnosti objektu bez mijenjanja njegove strukture, koristeći kompoziciju umjesto nasljeđivanja. Na primjer, ukoliko bi bilo potrebno dodati funkcionalnost naplate provizije za određene vrste računa ili kreirati specifične informacije za neke tipove transfera, Decorator to omogućava omotačima bez potrebe za mijenjanjem osnovne logike same transakcije.

5. Composite

Omogućuje da se grupe objekata i pojedinačni objekti tretiraju na isti način, organizirajući ih u hijerarhijske strukture stabla. Ukoliko korisnik u budućnosti bude posjedovao više povezanih računa (npr. tekući, devizni račun i štednju), Composite bi omogućio grupisanje ovih objekata. Klijentski kod bi tada mogao jednostavno zatražiti ukupno stanje nad cijelim stablom računa

jednako kao što bi zatražio stanje nad jednim pojedinačnim računom, bez pisanja kompleksnih petlji i provjera.

6. Bridge

Umjesto gomilanja podklasa zbog grananja koda u različitim smjerovima, Bridge pattern izoluje definiciju sistema od samog načina izvršavanja, čineći strukturu preglednijom. U slučaju našeg sistema, umjesto kreiranja potencijalnih kombinovanih klasa poput `StudentskiRacunSaFiksnomKamatom`, klasa `Racun` se definiše kao apstrakcija koja preko interfejsa `Kamata` koristi ispravnu logiku obračuna. Ovim Bridge pattern omogućava da se novi tipovi kamata i sličnih modifikacija dodaju nezavisno, bez potrebe za modifikacijom postojećih klasa računa.